Pathlib Module

Introduction

pathlib is a modern, object-oriented way to work with file and folder paths in Python. Instead of manually doing string path manipulation like **`"C:\\Users\\..."`** or **`"/home/user/..."`**, you use Path objects that work cleanly across operating systems.

Benefits

- Cleaner code than os.path
- Easy joining of paths using /
- Built-in methods for common tasks: create folders, list files, read/write text, rename, delete, etc.
- Cross-platform friendly (Windows/Linux/macOS)

Syntax

```
from pathlib import Path
```

Creating a Path

Python

```python
p1 = Path("data/file.txt")       # relative path

p2 = Path(r"C:\Users\Amar\doc") # Windows-style absolute path (raw string)

p3 = Path("/home/amar/doc")      # Linux/mac absolute path
```

Current working directory

Python

```python
from pathlib import Path

cwd = Path.cwd()
print(cwd)
```

Home directory

Python

```python
from pathlib import Path

home = Path.home()
print(home)
```

Joining paths

In **pathlib**, you usually **join** paths using **/ operator**.

Python

```python
from pathlib import Path

base = Path("project")
file_path = base / "data" / "input.txt"
print(file_path)
```

Output

```
project\data\input.txt
```

This **/** is **NOT** division here—it's **path joining**.

Checking path properties

Python

```python
p = Path("project/data/input.txt")

# Does it exist?
p.exists()

# Is it a file or folder?
p.is_file()
p.is_dir()

# Absolute path
p.resolve()   # full absolute path (also normalizes)
p.absolute()  # absolute without resolving symlinks (behavior differs)
```

Useful parts of a path

Python

```python
p = Path("project/data/input.txt")

print(p.name)        # input.txt
print(p.stem)        # input
print(p.suffix)      # .txt
print(p.suffixes)    # ['.txt'] (multiple suffixes possible: .tar.gz)
print(p.parent)      # project/data
print(p.parents)     # iterable of all parents
```

Changing file name / extension

Change name

Python

```
p = Path("data/input.txt")

new_p = p.with_name("output.txt")
```

Change suffix (extension)

Python

```
p = Path("data/input.txt")

new_p = p.with_suffix(".csv")
```

Listing files and folders

List immediate contents

Python

```
folder = Path("project")
for item in folder.iterdir():
    print(item)
```

List only files

Python

```
for item in folder.iterdir():
    if item.is_file():
        print(item.name)
```

Pattern searching

Find .txt files in one folder

Python

```
folder = Path("project")
for item in folder.glob("*.txt"):
    print(item)
```

Recursive search (all subfolders)

Python

```
for item in folder.rglob("*.txt"):
    print(item)
```

- **glob() = current folder search**
- **rglob() = deep recursive search**

Common patterns:

- **"*.py"** → all Python files
- **"data/*.csv"** → csv inside data folder
- **"**/*.json"** → recursive json (glob supports ** too)

Renaming & Deleting

Renaming

Python

```
p = Path("old.txt")
p.rename("new.txt")
```

Deleting

Python

```
Path("temp.txt").unlink()       # Delete a file
Path("empty_folder").rmdir()    # Delete empty folder
```

To delete a folder with contents, use **shutil. Rmtree()**

Working with relative vs absolute paths safely

If you want a file relative to your script location (common in projects):

Python

```
BASE_DIR = Path(__file__).resolve().parent
data_file = BASE_DIR / "data" / "input.txt"
```

- __file__ gives script file path
- .resolve().parent gives folder containing the script

Practice Set – 2

Download Link in **Description** and **Pinned Comment**

WATCH

Level up your coding with each episode in this focused Python series.

Next Video!

Practice Set - 2 Solution